

Ultron — Offline Multi-Agent Desktop Assistant: Engineering Blueprint

Sep 23, 2026 · @Someone

Architecture overview

Ultron is two processes and a database on one machine, with zero network egress: a Tauri shell hosting the React/R3F HUD, a Python orchestration service, Ollama, and PostgreSQL.

```

flowchart TD
    UI["Tauri shell<br/>React + R3F + Tailwind"] <--> WS["ws 127.0.0.1:8765"]
    API["FastAPI<br/>event bus"]
    API --> SUP["LangGraph<br/>supervisor"]
    SUP --> DEV["dev_agent"]
    SUP --> WEB["web_agent<br/>Playwright"]
    SUP --> OS["os_agent<br/>pyautogui"]
    SUP --> RAG["rag_agent<br/>CRAG loop"]
    DEV --> OLL["Ollama<br/>11434"]
    WEB --> OLL
    RAG --> PG["PostgreSQL<br/>pgvector"]
    SENS["Sensors<br/>whisper · Piper · MediaPipe"] --> API
  
```

Five principles that constrain everything downstream

P1 — Offline is an invariant, not a default. Playwright runs behind a proxy that blackholes anything outside an allowlist; the Tauri CSP forbids remote origins; fonts are bundled, not fetched. A dependency that needs the network at runtime is the wrong dependency.

P2 — One heavy model resident at a time. The supervisor is a small always-warm router. Workers are heavier and strictly transient. Every routing decision is therefore also a memory decision (§3).

P3 — One event bus, one direction of truth. The backend owns all state; the frontend renders events and computes none. Orb colour, node pulse and VRAM gauge are all subscriptions to the same `asyncio.Queue`, which keeps the 3D layer stateless and replayable.

P4 — Agents are isolated by capability, not by prompt. Each worker gets its own tool list and permissions. The dev agent cannot drive the mouse; the OS agent cannot query the vector store. Capability isolation is what makes an autonomous loop safe to leave running.

P5 — Fail visible. Every exception, tool timeout and model-load failure becomes an `error` event rendered in the HUD. A silent agent failure is indistinguishable from a hang, and hangs destroy trust faster than errors.

Request lifecycle

1. Hotkey or wake word → `faster-whisper` transcribes → `user_utterance` event.
2. Supervisor emits `graph_node{node:"supervisor", status:"active"}`, classifies intent, returns a route.
3. Runtime calls `ollama.ensure(model)` — unload previous worker, load new one, emitting `vram` throughout.
4. Worker runs tools, streaming `token` and `tool_call` events.
5. Worker returns; manager unloads it with `keep_alive=0`; supervisor continues or finishes.
6. Answer → Piper TTS → `audio_level` events drive the orb while speaking.

Steps 3 and 5 decide whether the system feels fast or broken, so they get their own section.

Repository layout

One monorepo, two independently runnable halves. The contract between them is `shared/events.schema.json` — generate the TypeScript types from it so a backend event rename breaks the frontend build rather than the demo.

```
ultron/
├── shared/
│   ├── events.schema.json          # single source of truth for the WS
├── envelope
│   └── gen/                        # codegen output: events.d.ts, events.py
├── frontend-tauri/
│   ├── src-tauri/
│   │   ├── tauri.conf.json        # window: transparent, decorations:false,
always-on-top
│   │   └── Cargo.toml
```

```

|   |   └─ src/main.rs           # global hotkey, tray, spawns/monitors
backend sidecar
|   └─ src/
|   |   └─ App.jsx               # Canvas + HUD composition root (§6)
|   |   └─ main.jsx
|   |   └─ three/
|   |       └─ Orb.jsx           # audio-reactive icosahedron + shader
|   |       └─ orb.glsl.js      # vertex/fragment source as template
strings
|   |   └─ AgentGraph.jsx       # 3D node graph of supervisor + workers
|   |   └─ AgentNode.jsx       # single node: sphere, label, emissive
pulse
|   |   └─ DataBeam.jsx         # animated edge when data flows
supervisor→worker
|   |   └─ Scene.jsx            # lights, bloom, postprocessing stack
|   |   └─ hud/
|   |       └─ HUD.jsx          # glassmorphism layout grid
|   |       └─ VramGauge.jsx
|   |       └─ ErrorConsole.jsx # tails error events, monospace, severity
colours
|   |   └─ ChatLog.jsx          # streaming token renderer
|   |   └─ StatusRail.jsx      # mic state, model loaded, latency
|   |   └─ hooks/
|   |       └─ useUltronSocket.js # WS client: reconnect, dispatch, send
|   |       └─ useAudioLevel.js  # Web Audio analyser → smoothed amplitude
ref
|   |   └─ useAgentState.js     # reducer over graph_node events
|   |   └─ store/ultronStore.js # zustand: single store, transient updates
|   |   └─ styles/index.css     # Tailwind entry + glass utility layer
|   └─ tailwind.config.js
|   └─ vite.config.js
|   └─ package.json
|
└─ backend-ai/
    └─ orchestrator.py          # FastAPI app + WS endpoint (§7)
    └─ config.py                # pydantic-settings, .env, model registry
    └─ bus.py                   # EventBus: publish/subscribe asyncio
queues
|   └─ graph/
|   |   └─ state.py             # UltronState TypedDict
|   |   └─ supervisor.py       # router node + conditional edges
|   |   └─ builder.py          # StateGraph assembly + checkpointer
|   |   └─ agents/

```

```

|   |   |   |— dev_agent.py
|   |   |   |— web_agent.py
|   |   |   |— os_agent.py
|   |   |   |— rag_agent.py      # retrieve → grade → rewrite → generate
|   |— llm/
|   |   |— ollama_manager.py      # load/unload, keep_alive tiers, VRAM
polling
|   |   |— prompts/              # one .md per agent, hot-reloadable
|   |   |— tools/
|   |   |   |— browser.py         # Playwright wrapper, persistent context
|   |   |   |— desktop.py        # pyautogui + subprocess, guarded
|   |   |   |— vision.py         # screenshot + crop + OCR for terminal
errors
|   |   |— registry.py           # capability allowlist per agent
|   |   |— memory/
|   |   |   |— db.py              # asyncpg pool
|   |   |   |— vectorstore.py     # pgvector upsert/search
|   |   |   |— ingest.py          # chunker + embedder CLI
|   |   |   |— schema.sql
|   |   |— sensors/
|   |   |   |— stt.py              # faster-whisper + Silero VAD loop
|   |   |   |— tts.py              # Piper streaming synth
|   |   |   |— gestures.py        # OpenCV + MediaPipe hand landmarks
|   |   |— pyproject.toml
|   |   |— .env.example
|
|— models/                        # gitignored: piper voices, whisper ct2,
gguf
|— scripts/
|   |— bootstrap.ps1 / bootstrap.sh
|   |— pull_models.py             # ollama pull + checksum verify
|— README.md

```

Why these boundaries

`tools/registry.py` exists so capability isolation is data, not discipline: each agent constructor takes a list of tool names and the registry refuses to hand back anything outside it.

`bus.py` sits between LangGraph and FastAPI deliberately. Sensors and the VRAM poller also publish to it, so the WebSocket layer has exactly one input and the graph has no idea a UI exists.

VRAM and model lifecycle

Tuned for the target machine: **GTX 1650 (4 GB VRAM), 16 GB system RAM**. The rule here is stricter than the usual supervisor-resident layout — the GPU holds exactly one tenant at a time, the active worker, and everything else runs on CPU. The 7 GB of “shared” VRAM Windows reports is system RAM reached over PCIe; Ollama does not allocate into it, and forcing the issue is slower than running on CPU outright. Plan against 4096 MB, minus roughly 500 MB the desktop compositor and a headful Chromium hold.

Model tiers

Tier	Role	Model	Quant	Footprint	Device	keep_alive
Router	intent classification, routing JSON, CRAG grading	qwen2.5:1.5b-instruct	Q4_K_M	1.1 GB	CPU	-1
Embedder	pgvector embeddings	nomic-embed-text	F16	0.3 GB	CPU	-1
STT	transcription	faster-whisper-base.en	int8	0.4 GB	CPU	resident
TTS	speech	Piper_en_GB-alba-medium	—	0.1 GB	CPU	resident
Worker	dev / web / os / RAG reasoning	qwen2.5-coder:3b	Q4_K_M	2.2 GB	GPU	0
Vision	terminal screenshots	moondream (1.8B)	Q4	1.8 GB	GPU, solo	0

GPU floor is zero between tasks and 2.2 GB during one, leaving ~1.3 GB of headroom for the compositor and Chromium. CPU-side residents total ~1.9 GB of your 16 GB.

Why the router goes on CPU. A 1.5B Q4 model emitting ~20 tokens of routing JSON takes about 1 s on six cores. Keeping it in VRAM instead would cost 1.1 GB permanently, which forces the worker down to a 1.5B model — and worker quality is what determines whether “find the S26 on Flipkart” actually completes. Trading 1 s of routing latency for a 3B worker is the single highest-value decision on this hardware.

Use one worker model, not three. With a 3 GB effective budget there is no room to specialise, and each swap costs 1–2 s. Run qwen2.5-coder:3b for dev, web and os, varying only the system prompt and tool list. llama3.2:3b is the alternative if you find the coder model weak at general instruction-following; test both against your own utterances before committing.

Force CPU placement per request with `options: {"num_gpu": 0}` — this is how the router and embedder stay off the GPU even though they run through the same Ollama daemon.

Ollama environment

Set these as machine-level environment variables before the daemon starts; per-request `keep_alive` overrides them but the defaults matter when something bypasses the manager.

```
OLLAMA_KEEP_ALIVE=0           # default: evict immediately
OLLAMA_MAX_LOADED_MODELS=2    # CPU router + one GPU worker; manager
                                enforces the GPU side
OLLAMA_NUM_PARALLEL=1         # no duplicated KV cache
OLLAMA_FLASH_ATTENTION=1      # Turing CC 7.5 supports it
OLLAMA_KV_CACHE_TYPE=q8_0     # halves KV cache; requires flash attention
OLLAMA_CONTEXT_LENGTH=4096    # 8k KV will not fit beside a 3B on 4 GB
OLLAMA_GPU_OVERHEAD=536870912 # reserve 512 MB for desktop + Chromium
OLLAMA_HOST=127.0.0.1:11434   # never bind 0.0.0.0
```

The 1650 has no tensor cores, so there is no fp16 path worth taking — stay on Q4_K_M throughout. `OLLAMA_GPU_OVERHEAD` is the one people skip and then spend an evening debugging: without it Ollama sizes the model against the full 4096 MB, then the compositor reclaims memory mid-generation and layers spill to CPU at random.

The manager contract

```
# backend-ai/llm/ollama_manager.py
import asyncio, httpx, pynvml
from contextlib import asynccontextmanager

HOST = "http://127.0.0.1:11434"
RESIDENT = {"qwen2.5:3b-instruct", "nomic-embed-text"}

class OllamaManager:
    def __init__(self, bus):
        self.bus = bus
        self._active: str | None = None
        self._lock = asyncio.Lock()          # serialises all worker swaps
        pynvml.nvmlInit()
        self._gpu = pynvml.nvmlDeviceGetHandleByIndex(0)
```

```

    async def _unload(self, model: str) -> None:
        async with httpx.AsyncClient(timeout=30) as c:
            await c.post(f"{HOST}/api/generate",
                        json={"model": model, "keep_alive": 0, "prompt":
""})

    async def _warm(self, model: str) -> None:
        ka = -1 if model in RESIDENT else "5m"
        async with httpx.AsyncClient(timeout=300) as c:
            await c.post(f"{HOST}/api/generate",
                        json={"model": model, "keep_alive": ka, "prompt":
""})

@asynccontextmanager
async def worker(self, model: str):
    """Guarantees at most one non-resident model in VRAM."""
    async with self._lock:
        if self._active and self._active != model:
            await self.bus.publish({"type": "model", "phase":
"unloading",
                                "model": self._active})
            await self._unload(self._active)
            self._active = None
        if self._active != model:
            await self.bus.publish({"type": "model", "phase": "loading",
                                "model": model})
            await self._warm(model)
            self._active = model
        await self.bus.publish({"type": "model", "phase": "ready",
                                "model": model})

    try:
        yield model
    finally:
        if model not in RESIDENT:
            await self._unload(model)
            self._active = None
            await self.bus.publish({"type": "model", "phase":
"evicted",
                                "model": model})

    async def telemetry(self, hz: float = 1.0) -> None:
        """Background task: pushes vram events to the HUD gauge."""
        while True:

```

```

m = pynvml.nvmlDeviceGetMemoryInfo(self._gpu)
await self.bus.publish({"type": "vram",
                        "used_mb": m.used // 1_048_576,
                        "total_mb": m.total // 1_048_576,
                        "active_model": self._active})

await asyncio.sleep(1 / hz)

```

Latency consequence, and the mitigation

A cold 7B Q4 load off NVMe costs roughly 1.5–3 s. That gap is dead air, so the UI has to own it: the orb shifts to the loading state and the target agent node in the graph pulses while `phase: "loading"` is in flight. Users tolerate a visible 2 s load; they do not tolerate 2 s of a frozen frame.

Two optional optimisations, in order of payoff:

1. **Speculative warm.** The supervisor emits its route decision before it finishes generating its rationale. Fire `_warm()` on that first token and you hide most of the load behind the router's own tail.
2. **Single worker model.** If VRAM is tight, collapse dev/web/os onto one 7B and vary only the system prompt and tool list. You lose some specialisation and gain zero swap time on the most common multi-hop tasks.

What a 7B costs you here. Ollama will load `qwen2.5-coder:7b Q4_K_M` by splitting layers — roughly 18 of 29 on the GPU, the rest on CPU — and you get 4–6 tokens/s. That is tolerable for a one-shot RAG answer and unusable for an agent loop that makes six tool calls. Keep 7B out of the graph; if you want it for long-form synthesis, expose it as an explicit "deep answer" command the user invokes knowingly.

Upgrade path. At 8 GB, move the router back onto the GPU with `keep_alive=-1` and raise the worker to 7B. At 12 GB or more, restore the three-worker specialisation from the original design and give the vision model `keep_alive="10m"`. Nothing in the architecture changes — only the tier table and two environment variables.

WebSocket event contract

One envelope, one endpoint (`ws://127.0.0.1:8765/ws`), JSON text frames. Define it once in `shared/events.schema.json` and generate both sides; a hand-maintained duplicate diverges within a week.

```
interface Envelope<T = unknown> {
  v: 1; // protocol version
  id: string; // uuid4, for ack/correlation
  ts: number; // epoch ms, backend clock
  type: EventType;
  run_id?: string; // LangGraph thread id, groups a whole task
  payload: T;
}
```

Backend → frontend

type	Payload	Drives
state	{phase: "idle" "listening" "thinking" "acting" "speaking" "error"}	Orb colour and pulse rate
audio_level	{rms: number} 0-1, ~30 Hz while speaking	Orb displacement during TTS
graph_node	{node: string, status: "active" "done" "failed", ms?: number}	Node graph highlight, edge beam
token	{text: string, role: "assistant", done: boolean}	Streaming chat log
tool_call	{agent, tool, args_preview, status}	Status rail, activity ticker
vram	{used_mb, total_mb, active_model}	VRAM gauge
model	{phase: "loading" "ready" "unloading" "evicted", model}	Loading shimmer on target node
error	{severity, source, message, traceback?}	Error console
transcript	{text, final: boolean}	Interim STT display
confirm_request	{action_id, description, risk}	Modal gate (§11)

type	Payload	Drives
screenshot	{action_id, png_b64, caption}	Dev agent evidence panel

Frontend → backend

type	Payload
user_utterance	{text, source: "voice" "text"}
confirm_response	{action_id, approved: boolean}
cancel	{run_id} — cooperative abort of the current graph run
mic	{enabled: boolean}
ping	{ } — every 10 s, expects pong

Transport rules

Backpressure. token and audio_level are high-frequency and lossy by design. The connection manager holds a bounded asyncio.Queue(maxsize=256) per client; on overflow it drops the oldest audio_level and vram frames and never drops state, error, graph_node or confirm_request. A dropped orb frame is invisible; a dropped confirmation gate is a bug with real consequences.

Reconnection. The frontend reconnects with exponential backoff capped at 5 s. On reconnect it sends {type:"resync"} and the backend replies with a single snapshot event carrying current phase, active node, last 50 chat messages and current VRAM. Never replay the event log — replaying graph_node events re-animates completed work and looks like a hallucination.

Ordering. One WebSocket, one asyncio task draining one queue per client. Do not parallelise the send loop; out-of-order state events cause orb flicker that is maddening to debug.

Auth. Bind to loopback only and require a token generated at Tauri startup, passed as ?token=. Any local process can open a loopback socket, and this one can drive your mouse.

Execution plan

Build the wire before the intelligence. Each phase has a done-when test you can run in under a minute; do not start the next phase until it passes.

P0 — Environment (half a day)

Install Python 3.11, Node 20 LTS, Rust stable + MSVC Build Tools, PostgreSQL 16, pgAdmin 4, Ollama, mpv. Create the venv, `npm create tauri-app`, set the environment variables from §3, and reboot so Ollama picks them up.

Then do the registry work now rather than at P5: run `schema.sql`, create `registry_apps`, `registry_media` and `registry_contacts`, and populate them by hand — ten apps, your contacts, and a music scan. Everything in P6 resolves through these tables, and having them early means you can test commands the moment the graph runs.

Done when: `ollama run qwen2.5:1.5b-instruct` responds, `psql -c "CREATE EXTENSION vector;"` succeeds, `SELECT count(*) FROM registry_apps` returns a non-zero number, and `nvidia-smi` reports 4096 MiB total.

P1 — The hollow bridge (1 day)

FastAPI with one WS endpoint that echoes a hardcoded `state` event every second. Tauri window with a React page that connects and prints envelopes to the console. No 3D, no LLM.

Done when: the Tauri window logs a backend heartbeat, and killing the backend triggers frontend reconnect with backoff.

P2 — The orb (2 days)

R3F canvas, icosahedron with the custom shader, `useUltronSocket` wired to the store, orb colour driven by the `state` event. Add `useAudioLevel` with `getUserMedia` → `AnalyserNode` and displace vertices by amplitude. Bloom via `@react-three/postprocessing`.

Done when: speaking into the mic visibly deforms the orb at 60 fps, and manually publishing `{"type": "state", "payload": {"phase": "thinking"}}` from a Python REPL changes its colour.

This is the phase to get right. Every later feature renders through it, and shader debugging is much harder once agent state is also in motion.

P3 — Supervisor skeleton (2 days)

LangGraph `StateGraph` with a supervisor node and three stub workers that return canned strings. Stream `astream_events` into the bus. Build `AgentGraph.jsx`: nodes in a ring around the orb, emissive pulse on `graph_node`.

Done when: typing "open my browser" lights the supervisor node, then the web node, then returns to idle — with no model doing real work yet.

P4 — Real models and VRAM discipline (2 days)

Swap the stubs for real inference under the split-device layout: router and embedder pinned to CPU, one 3B worker holding the GPU alone, evicted on return. Wire `telemetry()` to the VRAM gauge. Full walkthrough and code in the *Phase 4 build* section at the end of this document.

Done when: `ollama ps` shows the router at `100% CPU` permanently and the worker at `100% GPU` only during a task, dedicated VRAM never exceeds ~2.7 GB, and the gauge in the HUD returns to baseline within 2 s of the answer finishing.

P5 — Memory (2 days)

`schema.sql`, `asyncpg` pool, `ingest.py` over a folder of your own notes, HNSW index, the RAG agent with its retrieve → grade → rewrite → generate loop.

Done when: a question answerable only from your ingested notes returns a correct answer with source chunks, and a question with no local support triggers a visible CRAG rewrite rather than a confident fabrication.

P6 — Hands (3 days)

Playwright with a persistent context so logins survive. The `web_agent` with `goto`, `find_and_click`, `type_text`, `read_page`, `screenshot`. Then `desktop.py` with the confirmation gate. Then `dev_agent`: launch VS Code, screenshot the terminal pane, feed to the vision model.

Done when: "search for Samsung S26 on Flipkart" completes end to end and the top three results appear in the chat log; and "why is my terminal erroring" produces a screenshot in the evidence panel plus a plausible diagnosis.

Budget the most time here. Browser automation is where plans meet reality: selectors drift, modals interrupt, and pages load slower than any timeout you first guess.

P7 — Voice loop (2 days)

`faster-whisper` with Silero VAD in a worker thread, push-to-talk on a Tauri global hotkey first, wake word second. Piper TTS streaming to sounddevice, publishing `audio_level` per chunk.

Done when: a full spoken round trip — you speak, orb listens, agent acts, orb speaks while pulsing to Piper's output — runs without touching the keyboard.

P8 — Polish and hardening (ongoing)

Gesture control (MediaPipe pinch to mute, swipe to dismiss), error console filtering, `tauri build` producing a signed installer, bundled model checksums, crash recovery from the LangGraph checkpoint.

Done when: a fresh machine runs `bootstrap.ps1`, then the installer, and reaches a working orb with no manual steps beyond `ollama pull`.

Sequencing note

Estimates assume one engineer familiar with the stack. If you are learning R3F and LangGraph at once, double P2 and P3. The order matters more than the estimates: every phase after P1 is testable in isolation only because the bus exists first.

On this hardware, P4 is the phase that decides whether the project is viable. If the worker will not hold the GPU alone at a usable token rate, nothing later compensates — so do not move past it on a partial pass.

Frontend initialization code

Four files: the socket hook, the store, the orb, and `App.jsx`. The critical discipline is that nothing which changes at 60 fps ever touches React state — amplitude and colour are written into refs and consumed inside `useFrame`.

`src/store/ultronStore.js`

```
import { create } from "zustand";

export const useUltron = create((set) => ({
  phase: "idle",           // idle | listening | thinking | acting |
  speaking | error
  activeNode: null,
```

```

messages: [],
errors: [],
vram: { used_mb: 0, total_mb: 0, active_model: null },
connected: false,

apply: (evt) => set((s) => {
  switch (evt.type) {
    case "state":
      return { phase: evt.payload.phase };
    case "graph_node":
      return { activeNode: evt.payload.status === "active" ?
evt.payload.node : null };
    case "token": {
      const m = [...s.messages];
      const last = m[m.length - 1];
      if (last?.role === "assistant" && !last.done) {
        m[m.length - 1] = { ...last, text: last.text + evt.payload.text,
          done: evt.payload.done };
      } else {
        m.push({ role: "assistant", text: evt.payload.text, done:
evt.payload.done });
      }
      return { messages: m };
    }
    case "vram":
      return { vram: evt.payload };
    case "error":
      return { errors: [...s.errors.slice(-49), evt.payload] };
    default:
      return {};
  }
}),
}));

```

src/hooks/useUltronSocket.js

```

import { useEffect, useRef, useCallback } from "react";
import { useUltron } from "../store/ultronStore";

const URL = "ws://127.0.0.1:8765/ws";

export function useUltronSocket(token) {

```

```

const ws = useRef(null);
const retry = useRef(0);
const apply = useUltron((s) => s.apply);

useEffect(() => {
  let alive = true;
  let timer;

  const connect = () => {
    if (!alive) return;
    const sock = new WebSocket(`${URL}?token=${token}`);
    ws.current = sock;

    sock.onopen = () => {
      retry.current = 0;
      useUltron.setState({ connected: true });
      sock.send(JSON.stringify({ v: 1, type: "resync", payload: {} }));
    };

    sock.onmessage = (e) => {
      const evt = JSON.parse(e.data);
      if (evt.type === "audio_level") {
        // 30 Hz: bypass React entirely
        window.__ultronRms = evt.payload.rms;
        return;
      }
      apply(evt);
    };

    sock.onclose = () => {
      useUltron.setState({ connected: false });
      const wait = Math.min(5000, 250 * 2 ** retry.current++);
      timer = setTimeout(connect, wait);
    };

    sock.onerror = () => sock.close();
  };

  connect();
  return () => { alive = false; clearTimeout(timer); ws.current?.close(); };
}, [token, apply]);

```

```

const send = useCallback((type, payload = {}) => {
  if (ws.current?.readyState === WebSocket.OPEN) {
    ws.current.send(JSON.stringify({ v: 1, type, payload,
                                     id: crypto.randomUUID(), ts:
Date.now() }));
  }
}, []);

return { send };
}

```

src/three/0rb.jsx

An icosahedron displaced by 3D simplex noise, with amplitude and a phase-derived colour injected as uniforms. The colour lerps rather than snapping — an instant colour change reads as a glitch, a 300 ms lerp reads as intent.

```

import { useRef, useMemo } from "react";
import { useFrame } from "@react-three/fiber";
import * as THREE from "three";
import { useUltron } from "../store/ultronStore";

const PHASE_COLOR = {
  idle:      new THREE.Color("#1e6cff"),
  listening: new THREE.Color("#00e5ff"),
  thinking:  new THREE.Color("#a855f7"),
  acting:    new THREE.Color("#f59e0b"),
  speaking:  new THREE.Color("#22d3ee"),
  error:     new THREE.Color("#ef4444"),
};

const vertex = /* glsl */ `
  uniform float uTime;
  uniform float uAmp;
  varying float vNoise;

  // classic 3D simplex noise (Ashima) — trimmed for brevity
  vec3 mod289(vec3 x){return x-floor(x*(1.0/289.0))*289.0;}
  vec4 mod289(vec4 x){return x-floor(x*(1.0/289.0))*289.0;}
  vec4 permute(vec4 x){return mod289(((x*34.0)+1.0)*x);}
  vec4 taylorInvSqrt(vec4 r){return 1.79284291400159-0.85373472095314*r;}
  float snoise(vec3 v){

```

```

    const vec2 C=vec2(1.0/6.0,1.0/3.0); const vec4 D=vec4(0.0,0.5,1.0,2.0);
    vec3 i=floor(v+dot(v,C.yyy)); vec3 x0=v-i+dot(i,C.xxx);
    vec3 g=step(x0.yzx,x0.xyz); vec3 l=1.0-g;
    vec3 i1=min(g.xyz,l.zxy); vec3 i2=max(g.xyz,l.zxy);
    vec3 x1=x0-i1+C.xxx; vec3 x2=x0-i2+C.yyy; vec3 x3=x0-D.yyy;
    i=mod289(i);
    vec4 p=permute(permute(permute(
        i.z+vec4(0.0,i1.z,i2.z,1.0))+i.y+vec4(0.0,i1.y,i2.y,1.0))
        +i.x+vec4(0.0,i1.x,i2.x,1.0)));
    float n_=0.142857142857; vec3 ns=n_*D.wyz-D.xzx;
    vec4 j=p-49.0*floor(p*ns.z*ns.z);
    vec4 x_=floor(j*ns.z); vec4 y_=floor(j-7.0*x_);
    vec4 x=x_*ns.x+ns.yyyy; vec4 y=y_*ns.x+ns.yyyy; vec4 h=1.0-abs(x)-abs(y);
    vec4 b0=vec4(x.xy,y.xy); vec4 b1=vec4(x.zw,y.zw);
    vec4 s0=floor(b0)*2.0+1.0; vec4 s1=floor(b1)*2.0+1.0; vec4 sh=-
step(h,vec4(0.0));
    vec4 a0=b0.xzyw+s0.xzyw*sh.xxxy; vec4 a1=b1.xzyw+s1.xzyw*sh.zzww;
    vec3 p0=vec3(a0.xy,h.x); vec3 p1=vec3(a0.zw,h.y);
    vec3 p2=vec3(a1.xy,h.z); vec3 p3=vec3(a1.zw,h.w);
    vec4
norm=taylorInvSqrt(vec4(dot(p0,p0),dot(p1,p1),dot(p2,p2),dot(p3,p3)));
    p0*=norm.x; p1*=norm.y; p2*=norm.z; p3*=norm.w;
    vec4 m=max(0.6-vec4(dot(x0,x0),dot(x1,x1),dot(x2,x2),dot(x3,x3)),0.0);
m=m*m;
    return 42.0*dot(m*m,vec4(dot(p0,x0),dot(p1,x1),dot(p2,x2),dot(p3,x3)));
}

void main() {
    float n = snoise(normal * 1.8 + uTime * 0.25);
    vNoise = n;
    float displaced = 1.0 + n * (0.08 + uAmp * 0.55);
    vec3 pos = position * displaced;
    gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}
`;

const fragment = /* glsl */ `
    uniform vec3 uColor;
    uniform float uAmp;
    varying float vNoise;
    void main() {
        float rim = smoothstep(-0.6, 0.9, vNoise);
        vec3 col = uColor * (0.45 + rim * 1.3 + uAmp * 0.6);

```

```

    gl_FragColor = vec4(col, 0.82);
  }
`;

export default function Orb() {
  const mesh = useRef();
  const phase = useUltron((s) => s.phase);

  const uniforms = useMemo(() => ({
    uTime: { value: 0 },
    uAmp: { value: 0 },
    uColor: { value: PHASE_COLOR.idle.clone() },
  }), []);

  useFrame((state, delta) => {
    uniforms.uTime.value += delta;

    // amplitude: mic while listening, TTS level while speaking
    const target = phase === "speaking"
      ? (window.__ultronRms ?? 0)
      : (window.__ultronMicRms ?? 0);
    uniforms.uAmp.value += (target - uniforms.uAmp.value) * 0.18;

    // colour lerp - never snap
    uniforms.uColor.value.lerp(PHASE_COLOR[phase] ?? PHASE_COLOR.idle, delta
* 3.2);

    // thinking spins faster; idle barely breathes
    mesh.current.rotation.y += delta * (phase === "thinking" ? 0.55 : 0.12);
  });

  return (
    <mesh ref={mesh}>
      <icosahedronGeometry args={[1.25, 48]} />
      <shaderMaterial
        vertexShader={vertex}
        fragmentShader={fragment}
        uniforms={uniforms}
        transparent
        wireframe={false}

```

```

    />
  </mesh>
);
}

```

src/App.jsx

```

import { Canvas } from "@react-three/fiber";
import { EffectComposer, Bloom } from "@react-three/postprocessing";
import { OrbitControls } from "@react-three/drei";
import { Suspense } from "react";

import Orb from "../three/Orb";
import AgentGraph from "../three/AgentGraph";
import HUD from "../hud/HUD";
import { useUltronSocket } from "../hooks/useUltronSocket";
import { useAudioLevel } from "../hooks/useAudioLevel";
import { useUltron } from "../store/ultronStore";

export default function App() {
  const token = window.__TAURI_ULTRON_TOKEN__ ?? "dev";
  const { send } = useUltronSocket(token);
  useAudioLevel(); // writes window.__ultronMicRms
  const connected = useUltron((s) => s.connected);

  return (
    <div className="relative h-screen w-screen overflow-hidden bg-[#05070f]">
      <Canvas
        camera={{ position: [0, 0, 5.5], fov: 45 }}
        gl={{ antialias: true, alpha: true }}
        dpr={[1, 1.75]}
      >
        <color attach="background" args={["#05070f"]} />
        <ambientLight intensity={0.35} />
        <pointLight position={[4, 4, 4]} intensity={1.1} />

        <Suspense fallback={null}>
          <Orb />
          <AgentGraph radius={3.1} />
        </Suspense>

        <EffectComposer>

```

```

        <Bloom intensity={1.15} luminanceThreshold={0.22} mipmapBlur />
      </EffectComposer>

      <OrbitControls enablePan={false} enableZoom={false}
        autoRotate autoRotateSpeed={0.35} />
    </Canvas>

    <HUD send={send} />

    {!connected && (
      <div className="absolute inset-x-0 top-0 bg-red-500/15 backdrop-blur-
md
          border-b border-red-400/30 py-1.5 text-center
          text-xs tracking-wide text-red-200">
        backend offline – reconnecting
      </div>
    )}
  </div>
);
}

```

The glassmorphism layer

Define one Tailwind utility and reuse it everywhere rather than repeating blur classes:

```

/* src/styles/index.css */
@tailwind base; @tailwind components; @tailwind utilities;

@layer components {
  .glass {
    @apply rounded-2xl border border-white/10 bg-white/[0.04]
      backdrop-blur-xl shadow-[0_8px_32px_rgba(0,0,0,0.45)];
  }
  .glass-label {
    @apply text-[10px] uppercase tracking-[0.18em] text-cyan-200/50;
  }
}

```

HUD panels are `pointer-events-none` containers holding `pointer-events-auto` cards, so the canvas stays draggable in the gaps between panels. With Tauri's `decorations: false`, add `data-tauri-drag-region` to the top bar or the window becomes unmovable — an easy hour to lose.

Backend initialization code

`backend-ai/bus.py`

A fanout with bounded per-subscriber queues and a drop policy. This is what keeps a slow WebSocket from stalling the graph.

```
import asyncio, time, uuid
from typing import Any

LOSSY = {"audio_level", "vram", "token"}
CRITICAL = {"state", "error", "graph_node", "confirm_request"}

class EventBus:
    def __init__(self) -> None:
        self._subs: set[asyncio.Queue] = set()

    def subscribe(self) -> asyncio.Queue:
        q: asyncio.Queue = asyncio.Queue(maxsize=256)
        self._subs.add(q)
        return q

    def unsubscribe(self, q: asyncio.Queue) -> None:
        self._subs.discard(q)

    async def publish(self, event: dict[str, Any]) -> None:
        event.setdefault("v", 1)
        event.setdefault("id", str(uuid.uuid4()))
        event.setdefault("ts", int(time.time() * 1000))
        for q in list(self._subs):
            try:
                q.put_nowait(event)
            except asyncio.QueueFull:
                if event["type"] in CRITICAL:
                    _drop_one_lossy(q)          # make room, never drop
                else:
                    critical

                    try:
                        q.put_nowait(event)
                    except asyncio.QueueFull:
                        pass

    def _drop_one_lossy(q: asyncio.Queue) -> None:
        kept = []
```

```

dropped = False
while not q.empty():
    e = q.get_nowait()
    if not dropped and e["type"] in LOSSY:
        dropped = True
        continue
    kept.append(e)
for e in kept:
    q.put_nowait(e)

```

backend-ai/orchestrator.py

```

import asyncio, json, logging, secrets, traceback
from contextlib import asynccontextmanager

from fastapi import FastAPI, WebSocket, WebSocketDisconnect, Query, status
from langchain_core.messages import HumanMessage

from bus import EventBus
from config import settings
from graph.builder import build_graph
from llm.ollama_manager import OllamaManager
from memory.db import init_pool, close_pool

log = logging.getLogger("ultron")
bus = EventBus()
ollama = OllamaManager(bus)
RUNS: dict[str, asyncio.Task] = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.pool = await init_pool(settings.pg_dsn)
    app.state.graph = build_graph(bus, ollama, app.state.pool)
    # warm the resident router so the first utterance is not a cold start
    await ollama.warm_resident()
    telemetry = asyncio.create_task(ollama.telemetry(hz=1.0))
    try:
        yield
    finally:
        telemetry.cancel()
        for t in RUNS.values():

```

```

        t.cancel()
        await ollama.shutdown()          # evict every non-resident model
        await close_pool(app.state.pool)

app = FastAPI(lifespan=lifespan)

# — LangGraph → bus —————
async def run_graph(app: FastAPI, run_id: str, text: str) -> None:
    """Stream one graph invocation, translating LangGraph events to UI
    events."""
    cfg = {"configurable": {"thread_id": run_id}}
    state = {"messages": [HumanMessage(content=text)], "run_id": run_id}

    await bus.publish({"type": "state", "run_id": run_id,
                       "payload": {"phase": "thinking"}})

    try:
        async for ev in app.state.graph.astream_events(state, cfg,
version="v2"):
            kind = ev["event"]
            name = ev.get("name", "")

            if kind == "on_chain_start" and name in settings.graph_nodes:
                await bus.publish({"type": "graph_node", "run_id": run_id,
                                   "payload": {"node": name, "status":
"active"}})

            elif kind == "on_chain_end" and name in settings.graph_nodes:
                await bus.publish({"type": "graph_node", "run_id": run_id,
                                   "payload": {"node": name, "status":
"done"}})

            elif kind == "on_chat_model_stream":
                chunk = ev["data"]["chunk"].content
                if chunk:
                    await bus.publish({"type": "token", "run_id": run_id,
                                       "payload": {"text": chunk, "done":
False}})

            elif kind == "on_tool_start":
                await bus.publish({"type": "tool_call", "run_id": run_id,
                                   "payload": {"tool": name, "status":

```

```

"start",
                                "args_preview":
str(ev["data"].get("input"))[:160]}}})
        await bus.publish({"type": "state", "run_id": run_id,
                            "payload": {"phase": "acting"}})

        elif kind == "on_tool_end":
            await bus.publish({"type": "tool_call", "run_id": run_id,
                                "payload": {"tool": name, "status":
"end"}}})

            await bus.publish({"type": "token", "run_id": run_id,
                                "payload": {"text": "", "done": True}})

    except asyncio.CancelledError:
        await bus.publish({"type": "state", "run_id": run_id,
                            "payload": {"phase": "idle"}})
        raise
    except Exception as exc:
        log.exception("graph run failed")
        await bus.publish({"type": "error", "run_id": run_id,
                            "payload": {"severity": "fatal", "source":
"graph",
                                "message": str(exc),
                                "traceback": traceback.format_exc()
[-2000:]}}})
        await bus.publish({"type": "state", "payload": {"phase": "error"}})
    finally:
        RUNS.pop(run_id, None)
        await bus.publish({"type": "state", "payload": {"phase": "idle"}})

# — WebSocket endpoint —————
@app.websocket("/ws")
async def ws_endpoint(ws: WebSocket, token: str = Query(default="")):
    if not secrets.compare_digest(token, settings.session_token):
        await ws.close(code=status.WS_1008_POLICY_VIOLATION)
        return

    await ws.accept()
    q = bus.subscribe()

    async def pump() -> None:

```

```

        """Single drain task – guarantees ordering."""
        while True:
            evt = await q.get()
            await ws.send_text(json.dumps(evt))

    pump_task = asyncio.create_task(pump())

    try:
        while True:
            msg = json.loads(await ws.receive_text())
            mtype, payload = msg.get("type"), msg.get("payload", {})

            if mtype == "user_utterance":
                run_id = msg.get("id") or secrets.token_hex(8)
                RUNS[run_id] = asyncio.create_task(
                    run_graph(app, run_id, payload["text"]))

            elif mtype == "cancel":
                task = RUNS.get(payload.get("run_id", ""))
                if task:
                    task.cancel()

            elif mtype == "confirm_response":
                await approvals.resolve(payload["action_id"],
payload["approved"])

            elif mtype == "resync":
                await ws.send_text(json.dumps({
                    "v": 1, "type": "snapshot",
                    "payload": {"phase": "idle",
                                "vram": await ollama.snapshot(),
                                "active_model": ollama.active}}))

            elif mtype == "ping":
                await ws.send_text(json.dumps({"v": 1, "type": "pong",
payload": {}}))

    except WebSocketDisconnect:
        pass
    finally:
        pump_task.cancel()
        bus.unsubscribe(q)

```

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8765, log_level="info")
```

Two details worth flagging

`astream_events(version="v2")` is what makes the 3D graph animate without instrumenting every node by hand. It surfaces node entry and exit, model tokens and tool boundaries from the runtime itself, so agent code stays free of UI concerns.

The run task is deliberately not awaited inside the receive loop. If you await it, a long browser automation blocks the socket and `cancel` can never arrive — which is precisely when a user most wants to press stop.

LangGraph supervisor and sub-agents

```
stateDiagram-v2
    [*] --> supervisor
    supervisor --> dev_agent
    supervisor --> web_agent
    supervisor --> os_agent
    supervisor --> rag_agent
    dev_agent --> supervisor
    web_agent --> supervisor
    os_agent --> supervisor
    rag_agent --> supervisor
    supervisor --> [*] : FINISH
```

Workers always return to the supervisor. That single rule is what lets "find the S26 price on Flipkart and save it to my notes" chain a web hop into a RAG write without either agent knowing the other exists.

State

```
# graph/state.py
from typing import Annotated, Literal, TypedDict
from langgraph.graph.message import add_messages

AgentName = Literal["dev_agent", "web_agent", "os_agent", "rag_agent",
"FINISH"]

class UltronState(TypedDict):
    messages: Annotated[list, add_messages]
    next: AgentName
    run_id: str
    hops: int                # hard loop ceiling
    scratch: dict            # cross-agent handoff (urls, file paths,
                             screenshots)
```

Supervisor

The router is a small model doing one constrained job: emit JSON naming the next worker. Use structured output, not prose parsing — a 3B model produces reliable JSON under a tight schema and unreliable prose under any prompt.

```
# graph/supervisor.py
from pydantic import BaseModel, Field
from langchain_ollama import ChatOllama

MAX_HOPS = 8

class Route(BaseModel):
    next: AgentName = Field(description="worker to run, or FINISH when
complete")
    reason: str = Field(description="one short clause")

SYSTEM = """You route tasks to exactly one worker.
dev_agent: code, terminal errors, VS Code, debugging, screenshots of dev
tools.
web_agent: anything in a browser — search, shopping sites, web apps, forms.
os_agent:  native desktop apps, files, launching programs, keyboard/mouse.
rag_agent: questions about the user's own notes, documents, past
conversations.
FINISH:    the last worker already answered the user's request.
```

Return JSON only."""

```
def make_supervisor(bus, ollama):
    llm = ChatOllama(model="qwen2.5:3b-instruct", temperature=0,
                      keep_alive=-1).with_structured_output(Route)

    async def supervisor(state: UltronState) -> dict:
        if state.get("hops", 0) >= MAX_HOPS:
            return {"next": "FINISH"}
        route = await llm.aiinvoke([("system", SYSTEM), *state["messages"]])
        await bus.publish({"type": "tool_call", "run_id": state["run_id"],
                           "payload": {"agent": "supervisor", "tool":
"route",
                                     "args_preview": route.reason,
"status": "end"}}})
        return {"next": route.next, "hops": state.get("hops", 0) + 1}

    return supervisor
```

`MAX_HOPS` is not optional. An autonomous loop with a local 3B router will eventually ping-pong between two workers, and without a ceiling it does so until your GPU throttles.

Builder

```
# graph/builder.py
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver

def build_graph(bus, ollama, pool):
    g = StateGraph(UltronState)
    g.add_node("supervisor", make_supervisor(bus, ollama))
    g.add_node("dev_agent", make_dev_agent(bus, ollama))
    g.add_node("web_agent", make_web_agent(bus, ollama))
    g.add_node("os_agent", make_os_agent(bus, ollama))
    g.add_node("rag_agent", make_rag_agent(bus, ollama, pool))

    g.add_edge(START, "supervisor")
    g.add_conditional_edges(
        "supervisor",
        lambda s: s["next"],
        {"dev_agent": "dev_agent", "web_agent": "web_agent",
         "os_agent": "os_agent", "rag_agent": "rag_agent", "FINISH": END},
    )
```

```

for w in ("dev_agent", "web_agent", "os_agent", "rag_agent"):
    g.add_edge(w, "supervisor")

return g.compile(checkpointer=AsyncPostgresSaver(pool))

```

Worker pattern

Every worker is the same shape: acquire the model under the manager's context, run a ReAct loop with a restricted toolset, release. The `async with` is what enforces the one-worker-resident rule.

```

# graph/agents/web_agent.py
from langgraph.prebuilt import create_react_agent
from tools.registry import tools_for

def make_web_agent(bus, ollama):
    async def web_agent(state: UltronState) -> dict:
        async with ollama.worker("qwen2.5-coder:7b") as model:
            agent = create_react_agent(
                ChatOllama(model=model, temperature=0.1, keep_alive="5m"),
                tools=tools_for("web_agent"),      # goto, click, type, read,
                prompt=open("llm/prompts/web.md").read(),
            )
            out = await agent.ainvoke({"messages": state["messages"]})
            return {"messages": out["messages"][-1:]}
        return web_agent

```

Dev agent specifics

The dev agent's differentiator is sight. Its loop is: locate the VS Code window → screenshot the terminal panel region → OCR or vision-model read → diagnose.

```

# tools/vision.py (sketch)
async def capture_terminal() -> dict:
    win = find_window(title_contains="Visual Studio Code")
    shot = pyautogui.screenshot(region=terminal_region(win))    # bottom ~30%
    b64 = to_b64(shot)
    await bus.publish({"type": "screenshot",
                      "payload": {"png_b64": b64, "caption": "terminal"}})
    return {"image": b64}

```

Prefer a vision model over Tesseract for this. Terminal output is dense, coloured and often has ANSI artefacts; `qwen2.5v1:7b` reads a stack trace far more reliably than OCR, and it can reason about the error in the same pass.

RAG agent with the CRAG loop

```

flowchart LR
    Q[question] --> R[retrieve<br/>pgvector top-k]
    R --> G{grade<br/>relevant?}
    G -->|yes| GEN[generate<br/>with sources]
    G -->|no| RW[rewrite query]
    RW --> R2[retrieve again]
    R2 --> G2{still weak?}
    G2 -->|yes| ADM[admit gap<br/>offer web_agent]
    G2 -->|no| GEN

```

The grader is a separate cheap call on the resident router, scoring each chunk `yes / no` for relevance. Chunks that fail are discarded before generation rather than passed through with a hedge — the whole point of CRAG is that the generator never sees irrelevant context.

The corrective branch is capped at one rewrite. If the second retrieval still grades poorly, the agent says the local knowledge base does not cover it and hands back to the supervisor, which may route to `web_agent`. An offline assistant that admits a gap is more useful than one that pads an answer from parametric memory and presents it as your notes.

Memory layer

One database, four concerns: source documents, their embedded chunks, episodic memory of past runs, and LangGraph's checkpoints.

`memory/schema.sql`

```

CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;           -- hybrid lexical search

CREATE TABLE documents (
    id          BIGSERIAL PRIMARY KEY,
    source_path TEXT NOT NULL UNIQUE,
    title       TEXT,

```

```

    mime          TEXT,
    sha256        TEXT NOT NULL,                -- skip re-ingest when unchanged
    ingested_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE chunks (
    id            BIGSERIAL PRIMARY KEY,
    document_id   BIGINT REFERENCES documents(id) ON DELETE CASCADE,
    ordinal       INT NOT NULL,
    content       TEXT NOT NULL,
    token_count   INT,
    embedding     VECTOR(768) NOT NULL,         -- nomic-embed-text
    UNIQUE (document_id, ordinal)
);

-- episodic: what Ultron did, so it can answer "what did you do yesterday"
CREATE TABLE episodes (
    id            BIGSERIAL PRIMARY KEY,
    run_id        TEXT NOT NULL,
    agent         TEXT NOT NULL,
    summary       TEXT NOT NULL,
    embedding     VECTOR(768),
    outcome       TEXT CHECK (outcome IN ('ok','failed','cancelled')),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX chunks_hnsw ON chunks
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

CREATE INDEX chunks_trgm ON chunks USING gin (content gin_trgm_ops);
CREATE INDEX episodes_recent ON episodes (created_at DESC);

```

LangGraph's `AsyncPostgresSaver` creates its own checkpoint tables on first run — call `await saver.setup()` once during bootstrap and never hand-edit them.

Your corpus: three collections, not one

The planned content — personal facts, sample dialogues, Gita phrases — serves three different purposes and must not share a retrieval pool. Retrieving a Gita verse into a factual answer about your own notes is the failure that will make the system feel unreliable, and it happens on the first day if everything sits in one table.

```
ALTER TABLE chunks ADD COLUMN collection TEXT NOT NULL DEFAULT 'personal'
CHECK (collection IN ('personal','scripture','dialogue'));
CREATE INDEX chunks_collection ON chunks (collection);
```

Collection	Content	How it is used	Retrieved for facts?
personal	your notes, preferences, projects, people	normal RAG: retrieve → grade → cite	Yes
scripture	Gita verses	retrieved only when the question is about the Gita, or on an explicit quote request	No
dialogue	your sample exchanges	few-shot style examples injected into the system prompt, never into the context as evidence	Never

The `dialogue` collection is the one people get wrong. Those exchanges define how Ultron should *sound*; if they enter the context as retrieved documents, a 3B model will treat them as facts and answer questions with lines from an old conversation. Select the 2–3 nearest dialogues by embedding, put them in the system prompt under a `<style_examples>` header, and exclude the collection from `SEARCH` entirely.

Chunking the Gita. One chunk per verse, never split. Keep Sanskrit, transliteration and translation together in the same chunk with `chapter` and `verse` in metadata, so “what does 2.47 say” is a metadata lookup rather than a similarity guess. A verse is 60–120 tokens, well under the 400–600 target, and that is fine — forcing verses into uniform chunk sizes destroys the one clean boundary the text gives you.

Scale note. This corpus is small — likely 500 to 5,000 chunks. Below roughly 10,000 rows, a sequential scan with cosine distance returns in under 5 ms and HNSW buys you nothing. Create the index anyway so it is there when the corpus grows, but do not spend time tuning `ef_search`. What matters at this scale is the grader: with a small pool, top-k always returns *something*, and only the relevance grade stops a loosely related verse from being presented as an answer.

Action registries

The same database holds what Ultron is allowed to *do*. These are not vector collections — they are lookup tables the OS agent resolves against, described in §11.

```
CREATE TABLE registry_apps (
  id          BIGSERIAL PRIMARY KEY,
  name        TEXT NOT NULL,          -- "Visual Studio Code"
  aliases     TEXT[] NOT NULL DEFAULT '{}', -- {vscode, code, editor}
  launch_cmd  TEXT NOT NULL,          -- "code" | path | protocol uri
  process_name TEXT,                  -- "Code.exe" – for closing
  protected   BOOLEAN NOT NULL DEFAULT false
);

CREATE TABLE registry_media (
  id          BIGSERIAL PRIMARY KEY,
  title       TEXT NOT NULL,
  artist      TEXT,
  path        TEXT NOT NULL,
  tags        TEXT[] DEFAULT '{}',      -- {favourite, focus, bhajan}
  embedding   VECTOR(768)
);

CREATE TABLE registry_contacts (
  id          BIGSERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  aliases     TEXT[] DEFAULT '{}',
  phone       TEXT,                    -- E.164, no spaces
  channel     TEXT CHECK (channel IN ('whatsapp','sms','email'))
);
```

Create these in P0, not P5. Every "do what I command" feature resolves through them, and populating them is a twenty-minute job that removes most of the guesswork a 3B model would otherwise have to do.

Retrieval

Hybrid beats pure vector on a personal knowledge base, because you search your own notes by remembered exact strings (a filename, an error code, a person's name) as often as by meaning.

```
SEARCH = """
WITH vec AS (
  SELECT id, content, document_id,
         1 - (embedding <=> $1) AS score
  FROM chunks ORDER BY embedding <=> $1 LIMIT 20
),
lex AS (
  SELECT id, content, document_id,
         similarity(content, $2) AS score
  FROM chunks WHERE content %% $2 ORDER BY score DESC LIMIT 20
)
SELECT DISTINCT ON (id) id, content, document_id, score
FROM (SELECT * FROM vec UNION ALL SELECT * FROM lex) u
ORDER BY id, score DESC;
"""
```

Rank-fuse the two lists in Python (reciprocal rank fusion, k=60) and pass the top 5 to the grader. Retrieving 20 and grading down to 5 costs one cheap router call and removes most of the noise that makes local RAG feel unreliable.

Tuning that actually matters

Setting	Value	Why
hnsw.ef_search	64-100	Per-session recall/latency dial; raise if answers miss known chunks
maintenance_work_mem	2 GB	HNSW build is dramatically slower below this
shared_buffers	25% of RAM	Standard, but easy to leave at the 128 MB default
Chunk size	400-600 tokens, 15% overlap	Small enough to grade, large enough to answer
max_connections	20	One desktop app; the pool needs 5

pgAdmin 4 workflow

Register the local server once, then keep three saved queries in the query tool: chunk count by document, the last 20 episodes, and an index-usage check (`pg_stat_user_indexes`). When RAG answers go strange, the cause is almost always ingestion — a document that failed to chunk, or a duplicate ingest doubling similar vectors — and those three queries tell you which within a minute.

Run `VACUUM ANALYZE chunks`; after every bulk ingest. pgvector's planner choices degrade noticeably on stale statistics.

Sensory I/O

All three sensors run in their own threads and publish to the bus through `asyncio.run_coroutine_threadsafe`. None of them touch the graph directly.

Speech to text

```
# sensors/stt.py
from faster_whisper import WhisperModel
import sounddevice as sd, numpy as np, torch

model = WhisperModel("small.en", device="cuda", compute_type="int8_float16")
vad, utils = torch.hub.load("snakers4/silero-vad", "silero_vad",
                             trust_repo=True, source="local") # vendored
offline

SAMPLE_RATE = 16_000
SILENCE_MS = 700 # end-of-utterance threshold

def listen_loop(bus, loop):
    buf, speaking, silence = [], False, 0
    with sd.InputStream(samplerate=SAMPLE_RATE, channels=1, dtype="float32",
                       blocksize=512) as stream:
        while True:
            block, _ = stream.read(512)
            mono = block[:, 0]

            rms = float(np.sqrt(np.mean(mono ** 2)))
            publish(bus, loop, {"type": "audio_level",
                                "payload": {"rms": min(1.0, rms * 12)}}})
```

```

is_speech = vad(torch.from_numpy(mono), SAMPLE_RATE).item() > 0.5
if is_speech:
    if not speaking:
        publish(bus, loop, {"type": "state",
                             "payload": {"phase": "listening"}})
        speaking, silence = True, 0
        buf.append(mono)
    elif speaking:
        silence += 32 # ms per 512 frames @16k
        buf.append(mono)
        if silence >= SILENCE_MS:
            audio = np.concatenate(buf)
            segs, _ = model.transcribe(audio, beam_size=1,
                                       vad_filter=False,
                                       language="en")

            text = "".join(s.text for s in segs).strip()
            if text:
                publish(bus, loop, {"type": "transcript",
                                    "payload": {"text": text,
                                                "final": True}})

            buf, speaking, silence = [], False, 0

```

On a 4 GB card, run Whisper on CPU: `WhisperModel("base.en", device="cpu", compute_type="int8", cpu_threads=6)`. It holds ~0.4 GB of RAM and transcribes a 5 s utterance in roughly 0.8 s, which is faster than the round trip you are about to make anyway. Putting it on the GPU would compete directly with the worker for the only 2.2 GB you have. `small.en` on CPU is about 2 s per utterance — noticeably laggy in a voice loop, so start with `base.en` and only move up if accuracy on your own speech is poor.

Start with push-to-talk on a Tauri global hotkey; add an always-on wake word (`openWakeWord`, also CPU) only after the rest is stable, because a false wake during a `pyautogui` action is genuinely disruptive.

Text to speech

```
# sensors/tts.py
from piper import PiperVoice
import sounddevice as sd, numpy as np

voice = PiperVoice.load("models/piper/en_GB-alba-medium.onnx")

async def speak(bus, text: str):
    await bus.publish({"type": "state", "payload": {"phase": "speaking"}})
    with sd.OutputStream(samplerate=voice.config.sample_rate,
                        channels=1, dtype="int16") as out:
        for chunk in voice.synthesize_stream_raw(text):
            pcm = np.frombuffer(chunk, dtype=np.int16)
            out.write(pcm)
            rms = float(np.sqrt(np.mean((pcm / 32768.0) ** 2)))
            await bus.publish({"type": "audio_level", "payload": {"rms":
rms}})
    await bus.publish({"type": "state", "payload": {"phase": "idle"}})
```

Streaming per sentence rather than per paragraph is what makes the orb feel alive while speaking — it starts pulsing within ~150 ms of the first token instead of after the whole answer synthesises. Piper is CPU-only, so it costs no VRAM and runs happily alongside a loaded worker.

Gestures

MediaPipe Hands at 15 fps on a 480p capture is plenty and leaves the GPU alone. Keep the gesture vocabulary tiny — three reliable gestures beat nine unreliable ones:

Gesture	Action
Open palm held 500 ms	Toggle mic on/off
Pinch (thumb + index)	Cancel current run
Swipe left	Dismiss the error console

Debounce every gesture with a 1 s cooldown and require the pose to persist across 5 consecutive frames. Without both, you will cancel runs by scratching your nose.

Publish a `gesture` event to the bus rather than calling actions directly, so the frontend can show a brief confirmation flash. A gesture that fires invisibly is indistinguishable from a crash.

Automation and guardrails

This section is the one to over-engineer. Everything else fails by being wrong; this fails by doing something.

Playwright

One persistent context for the whole session so logins survive, launched once at startup rather than per task.

```
# tools/browser.py
from playwright.async_api import async_playwright

class Browser:
    async def start(self):
        self._pw = await async_playwright().start()
        self.ctx = await self._pw.chromium.launch_persistent_context(
            user_data_dir="./.pwprofile",
            headless=False,                                # visible: the user must see
            viewport={"width": 1440, "height": 900},
            args=["--disable-blink-features=AutomationControlled"],
        )
        self.page = self.ctx.pages[0] if self.ctx.pages else await
        self.ctx.new_page()

    async def read_page(self) -> str:
        """Accessibility snapshot, not raw HTML – 10x fewer tokens, better
        grounding."""
        snap = await self.page.accessibility.snapshot()
        return flatten(snap)[:6000]

    async def find_and_click(self, description: str):
        for role in ("button", "link", "textbox"):
            loc = self.page.get_by_role(role, name=description)
```

```

    if await loc.count():
        await loc.first.click(timeout=5000)
        return f"clicked {role}: {description}"
    raise LookupError(f"no element matching {description!r}")

```

Give the model the accessibility tree, not the DOM. Raw HTML from a site like Flipkart is tens of thousands of tokens of wrapper divs; the accessibility snapshot is a few hundred lines of roles and names that a 7B model can actually reason over.

Worked example — "Search for Samsung S26 on Flipkart": `goto(flipkart.com)` → `read_page` → `find_and_click("Search for products")` → `type_text("Samsung S26")` → `press("Enter")` → wait for network idle → `read_page` → extract the top three titles and prices → return structured JSON. Six tool calls, each one an event the HUD renders. Expect to iterate on the dismissal of login modals; that is the step that breaks, not the search.

Desktop control

`pyautogui` is the blunt instrument. Prefer `subprocess` for anything that has a CLI — `code .` beats clicking the VS Code icon, and `start whatsapp://send?phone=...` beats navigating a GUI.

```

pyautogui.FAILSAFE = True      # mouse to a screen corner aborts everything
pyautogui.PAUSE = 0.25        # human-visible pacing; also survives slow UI
                                redraws

```

Command surface

The model never guesses. A 3B model asked to "open my editor" will invent a path and fail. Instead every actionable thing resolves through the registry tables from §9: the agent calls `resolve(kind, phrase)`, gets back up to three candidate rows, and executes a typed action against a row it did not invent. One confident match executes; two close matches ask which; none says so plainly.

```
# tools/resolve.py
from rapidfuzz import process, fuzz

async def resolve(pool, kind: str, phrase: str, limit: int = 3) ->
list[dict]:
    table = {"app": "registry_apps", "media": "registry_media",
            "contact": "registry_contacts"}[kind]
    rows = await pool.fetch(f"SELECT * FROM {table}")
    hay = {r["id"]: " ".join([r["name"], *(r.get("aliases") or [])]) for r in
rows}
    hits = process.extract(phrase, hay, scorer=fuzz.WRatio, limit=limit)
    return [dict(next(r for r in rows if r["id"] == rid), score=score)
            for _, score, rid in hits if score > 60]
```

Fuzzy matching on aliases handles the realistic failure modes — mishearing, abbreviation, your own inconsistent naming — without a model call. Fall back to embedding similarity on `registry_media` only, where "play something calm" has no lexical overlap with any title.

Opening and closing applications

Command	Implementation	Gate
"open X"	<code>os.startfile(row.launch_cmd)</code>	Auto
"close X"	<code>taskkill /IM {process_name} (no /F)</code>	Auto
"force close X"	<code>taskkill /F /IM {process_name}</code>	Confirm
"close everything"	enumerate, subtract protected set, list them, then close	Confirm, with the list

Graceful close first, always. `taskkill` without `/F` sends `WM_CLOSE`, so an editor with unsaved work prompts you rather than losing it. Only escalate to `/F` on an explicit second instruction.

The protected set is non-negotiable and hard-coded, not a database column you might edit wrong: `explorer.exe`, `ollama.exe`, `postgres.exe`, the Python backend, and the Tauri app itself. "Close everything" that kills Ollama and Postgres leaves you with a dead assistant and no way to tell it to recover.

```

PROTECTED = {"explorer.exe", "ollama.exe", "ollama app.exe",
             "postgres.exe", "python.exe", "ultron.exe",
             "msedgewebview2.exe"}

async def close_all(bus, dry_run_first: bool = True):
    targets = [p for p in visible_windows() if p.name.lower() not in
PROTECTED]
    ok = await approvals.require(
        bus, f"Close {len(targets)} apps: " + ", ".join(t.title for t in
targets),
        risk="destructive")
    if not ok:
        return "cancelled"
    for t in targets:
        subprocess.run(["taskkill", "/IM", t.name], capture_output=True)

```

Enumerate visible top-level windows rather than all processes. Killing by process list catches background services and tray helpers you never meant to touch.

Music

Local files only — a cloud music service would break the offline invariant, and the local path is better anyway because you control the library.

Index once with a scan script that walks your music folder, reads tags with `mutagen`, and writes `registry_media` rows with an embedding of `title + artist + tags`. Mark favourites with a `favourite` tag; "play my favourite music" then becomes a tag query and a shuffle, not a guess.

Play through `mpv` with its JSON IPC socket, which gives you real transport control instead of fire-and-forget:

```
MPV_PIPE = r"\\.\pipe\ultron-mpv"

def start_player():
    subprocess.Popen(["mpv", "--idle=yes", "--no-video",
                      f"--input-ipc-server={MPV_PIPE}"])

def mpv(cmd: list):
    with open(MPV_PIPE, "r+b", buffering=0) as p:
        p.write(json.dumps({"command": cmd}).encode() + b"\n")

# tools exposed to os_agent
play      = lambda path: mpv(["loadfile", path, "replace"])
queue     = lambda path: mpv(["loadfile", path, "append"])
pause     = lambda: mpv(["set_property", "pause", True])
next_track= lambda: mpv(["playlist-next"])
volume    = lambda v: mpv(["set_property", "volume", v])
```

One long-lived `mpv` process started with the backend costs about 40 MB of RAM and no VRAM. Duck the volume to 25% while Piper speaks and restore it afterwards — without that, TTS and music fight each other and the whole thing feels broken.

WhatsApp

There is no local API, so there are two routes and they are not equivalent.

Route	Can send	Can read incoming	Cost
<code>whatsapp://</code> protocol handler	Composes, you press Enter	No	Trivial, robust
WhatsApp Web via Playwright	Yes	Yes	Selector maintenance, session drift

Start with the protocol handler — it is two lines and it never breaks:

```
os.startfile(f"whatsapp://send?phone={contact['phone']}&text={quote(msg)}")
```

It opens the chat with your message pre-filled and stops. That is the right default: it lines up with the confirm gate, and the send is a human keystroke.

If you want Ultron to *read* messages and draft replies, use WhatsApp Web in the same persistent Playwright context from §11. Scrape the unread list from the accessibility tree, draft with the worker model, and route the send through `approvals.require` so nothing leaves your account without a click. Two cautions worth knowing up front: automating the web client can get a session logged out or flagged, so keep the pacing human and the volume low; and the conversation text you scrape is untrusted input — a message containing instructions must never reach the agent as a command. The delimiter rule under *Prompt injection* below covers this, and WhatsApp is exactly the case it exists for.

When Ultron does not know a command

The default for an unrecognised instruction is to say so, not to improvise with `pyautogui`. Add the tool but constrain the prompt: *if no registry entry matches and no tool applies, reply that you cannot do it yet and suggest the registry row to add*. An assistant that tells you to add "Spotify" to `registry_apps` is far more useful than one that blindly clicks at coordinates and reports success.

The confirmation gate

Classify every tool at registration time and gate the destructive ones on a round trip to the UI.

Risk	Examples	Policy
Read	screenshot, read_page, vector search	Auto
Navigate	goto, scroll, launch an app	Auto
Write-local	create a file, type into an editor	Auto, logged
Send	post a message, submit a form, send an email	Confirm
Destructive	delete, overwrite, purchase, change settings	Confirm , with a diff

```
# tools/approvals.py
_pending: dict[str, asyncio.Future] = {}

async def require(bus, description: str, risk: str, timeout: float = 60) ->
bool:
    aid = secrets.token_hex(6)
    fut = asyncio.get_running_loop().create_future()
    _pending[aid] = fut
```

```

    await bus.publish({"type": "confirm_request",
                      "payload": {"action_id": aid,
                                "description": description, "risk":
risk}}})
    try:
        return await asyncio.wait_for(fut, timeout)
    except asyncio.TimeoutError:
        return False # silence is refusal, never consent
    finally:
        _pending.pop(aid, None)

async def resolve(action_id: str, approved: bool) -> None:
    fut = _pending.get(action_id)
    if fut and not fut.done():
        fut.set_result(approved)

```

Timeout defaults to denial. An autonomous agent that proceeds because nobody answered is the failure mode that turns a useful assistant into an incident.

Prompt injection

A web page the agent reads is untrusted data, never instructions. Text scraped from a site can and will contain lines addressed to your model. Three defences, in order of importance:

1. Wrap all page content in delimiters and instruct the agent that content inside them describes the page and never commands it.
2. Keep the confirmation gate outside the model's reach — an injected instruction cannot approve its own action because approval arrives over the WebSocket from a human click.
3. Log every tool call with its arguments to `episodes`, so an unexplained action is reconstructable after the fact.

Kill switch

Three layers, because one is never enough: `pyautogui.FAILSAFE` corner abort, a global hotkey that cancels the run task and closes the browser context, and the tray item that terminates the sidecar. Test all three before you first leave the system running unattended.

Hardware, testing and risks

Minimum viable machine

Target machine: GTX 1650 4 GB, 16 GB RAM. RAM, not VRAM, is the binding constraint once everything is running.

Consumer	RAM	Note
Windows + background	3.5 GB	Assume 4 GB with a browser and Explorer open
Python backend	2.5 GB	whisper + MediaPipe + Playwright driver + asyncpg
Ollama CPU residents	1.9 GB	router 1.1 + embedder 0.3 + model overhead
Chromium (headful, Playwright)	1.5 GB	Grows with tab count; cap at 3 tabs
PostgreSQL	1.2 GB	With <code>shared_buffers = 1GB</code> , not the 25% default
Tauri + WebView2	0.6 GB	Production build; the Vite dev server adds ~1 GB more
Total	~11.2 GB	~4.8 GB slack on 16 GB

That slack disappears fast. Three rules keep it from becoming swap thrash, which on this configuration is far more painful than any GPU limit:

- Set `shared_buffers = 1GB` and `maintenance_work_mem = 512MB` in `postgresql.conf` . The §9 defaults assume a 32 GB machine and will starve everything else. Build the HNSW index once with `maintenance_work_mem = 2GB` set for that session only, then drop it back.
- Do not run `vite dev` and the packaged Tauri app at the same time in daily use. Develop, build, close the dev server.
- Run MediaPipe only while the gesture feature is on, and default it off. A 480p capture loop holds ~400 MB and a camera handle for a feature you will use rarely.

On the GPU side, nothing exceeds 2.2 GB at any moment, so OOM is not the failure mode you will hit — layer spillover is. Watch for generation dropping to single-digit tokens per second: that means Ollama sized a model against VRAM that the compositor then took

back.

Testing

Layer	Approach
Event contract	JSON-schema validate every published event in dev; a contract break should fail loudly, not silently mis-render
Graph routing	30 fixed utterances → asserted route, run on every prompt edit; prompt changes regress routing constantly
VRAM discipline	Integration test that runs three tasks and asserts <code>ollama ps</code> returns to router-only between each
Browser tools	Pin a local static-site fixture for selector tests; keep live-site runs manual
Frontend	React Testing Library for the HUD; for R3F, assert store→uniform mapping in isolation rather than rendering the canvas
End to end	A weekly manual script of the five tasks you actually use

Risks

Risk	Likelihood	Mitigation
Small router mis-routes multi-step tasks	High	Few-shot the routing prompt with your own real utterances; log every route to <code>episodes</code> and review weekly
Model swap latency feels sluggish	High	Speculative warm on first route token; collapse to a single worker model if it persists
Playwright selectors break on site redesigns	High	Accessibility-tree targeting over CSS selectors; treat the web agent as best-effort, never a dependency

Risk	Likelihood	Mitigation
7B model produces malformed tool JSON	Medium	Structured output with pydantic, one retry with the validation error appended, then fail visibly
Shader or bloom tanks framerate on integrated GPUs	Medium	<code>dpr=[1, 1.75]</code> , bloom toggle in settings, fallback to a flat-shaded orb
<code>pyautogui</code> acts on the wrong window	Medium	Verify the foreground window title before every click; confirmation gate on anything destructive
Whisper + worker + embedder exceed VRAM together	Medium	Count STT into the resident floor; drop to <code>base.en</code> on 8 GB cards
Tauri sidecar orphaned on crash	Low	PID file plus a startup sweep that kills stale backends

If the timeline compresses

Cut in this order, from first to cut:

1. Gestures (P8) — the highest effort-to-value ratio in the plan.
2. Wake word — push-to-talk is nearly as good and far more reliable.
3. The 3D node graph — keep the orb, render agent state as HUD chips. The orb carries almost all of the perceived intelligence.
4. The dev agent's vision loop — text-only error paste gets 80% of the value.

Do not cut: the event bus, the VRAM manager, or the confirmation gate. Each is close to impossible to retrofit, and the first two are what the whole design rests on.

Windows specifics

The GDDR6 card is the good one. 192 GB/s against the GDDR5 variant's 128 GB/s. On a 2.2 GB working set that puts `qwen2.5-coder:3b Q4_K_M` at roughly 35–45 tokens/s — comfortably interactive, and enough that a six-hop agent loop finishes in well under a minute once model swap is out of the path.

Reclaim VRAM from the desktop. WDDM reserves memory before Ollama sees any, and the largest avoidable consumer is Chromium's compositor. Launch Playwright with GPU acceleration off:

```
args=["--disable-blink-features=AutomationControlled",
      "--disable-gpu", "--disable-software-rasterizer"]
```

That returns roughly 250–350 MB and costs nothing — the agent reads the accessibility tree, not pixels. Do the same in Edge or Chrome if you keep one open while working. Task Manager's "Shared GPU memory" figure is not usable VRAM; only "Dedicated GPU memory" matters.

Window targeting. `pyautogui` has no window model on Windows; pair it with `pygetwindow` and verify before every click:

```
import pygetwindow as gw

def focus(title_contains: str, timeout: float = 3.0):
    for w in gw.getWindowsWithTitle(title_contains):
        if w.isMinimized:
            w.restore()
        w.activate()
        time.sleep(0.4)                                # WDDM redraw; clicking early
misses                                                misses
        if title_contains.lower() in (gw.getActiveWindow().title or
        "").lower():
            return w
        raise RuntimeError(f"window {title_contains!r} not focused")
```

The active-window assertion is what stops a mis-timed click landing in whatever you happened to have open. Treat it as mandatory, not defensive.

Launching things. Prefer protocol handlers and CLIs over GUI navigation:

Target	Command
VS Code in a folder	<code>subprocess.run(["code", path], shell=True)</code> — needs <i>Add to PATH</i> at install
WhatsApp message	<code>os.startfile(f"whatsapp://send?phone=91{number}&text={quote(msg)}")</code> — WhatsApp Desktop must be installed; it opens the chat pre-filled but does not send, which is the correct default
Any registered app	<code>os.startfile(path)</code>

Target	Command
URL in the agent's browser	Playwright <code>goto</code> , never <code>os.startfile</code> — otherwise it lands in your personal browser session

The WhatsApp handler stopping short of sending is a feature: it lines up with the confirmation gate in §11, and the user presses Enter.

Build and tooling notes. Tauri needs the MSVC Build Tools and WebView2 Runtime (preinstalled on Windows 11). `scripts/bootstrap.sh` is dead weight — write only the `.ps1`. Piper ships a Windows binary; call it via `subprocess` if the Python bindings give you trouble. `sounddevice` defaults to WASAPI and is fine, but set an explicit device index rather than trusting the default, which changes when you plug in headphones.

Postgres on Windows does not scale `shared_buffers` the way it does on Linux; 1 GB is the practical ceiling and going higher will not help. Run it as a service so it survives a backend crash.

Open questions to resolve before P4

- ~~Which GPU?~~ GTX 1650, 4 GB GDDR6. §3 and the tables above are tuned to it.
- ~~Windows or Linux?~~ Windows. See *Windows specifics* above.
- What goes into the knowledge base first? Ingestion format drives the chunker, and the chunker is awkward to change after you have embedded a few thousand chunks.
- Which five tasks do you actually want this for? The plan currently optimises for the three you named — Flipkart search, WhatsApp messaging, terminal debugging. If the real daily use is something else, P6's tool list should change before you build it.

Phase 4 build

This phase replaces the P3 stubs with real inference on the split-device layout. It is the viability gate for the whole project on a 4 GB card: if the worker cannot hold the GPU alone at a usable token rate, nothing later fixes that.

Before starting, P3 must pass — the node graph should already light up from stubbed workers. Debugging routing and VRAM at the same time is miserable.

Step 1 — Pull the models

```
ollama pull qwen2.5:1.5b-instruct    # router, CPU
ollama pull nomic-embed-text        # embedder, CPU
ollama pull qwen2.5-coder:3b        # worker, GPU
ollama pull moondream               # vision, GPU, solo (optional at P4)

ollama list                          # confirm sizes: 1.0 / 0.27 / 1.9 / 1.7
GB
```

Step 2 — config.py

Device placement belongs in configuration, not scattered through agent code. One table decides what runs where.

```
# backend-ai/config.py
from pydantic_settings import BaseSettings
from pydantic import Field

class Settings(BaseSettings):
    pg_dsn: str = "postgresql://ultron:ultron@127.0.0.1:5432/ultron"
    ollama_host: str = "http://127.0.0.1:11434"
    session_token: str = ""

    router_model: str = "qwen2.5:1.5b-instruct"
    worker_model: str = "qwen2.5-coder:3b"
    vision_model: str = "moondream"
    embed_model: str = "nomic-embed-text"

    ctx_router: int = 2048          # routing JSON needs very little
    ctx_worker: int = 4096

    vram_total_mb: int = 4096
    vram_reserve_mb: int = 700     # compositor + Chromium; do not allocate
    into this

    graph_nodes: set[str] = Field(default_factory=lambda: {
        "supervisor", "dev_agent", "web_agent", "os_agent", "rag_agent"})

    class Config:
        env_file = ".env"
```

```

settings = Settings()

# device placement – the single source of truth
CPU_MODELS = {settings.router_model, settings.embed_model}
GPU_MODELS = {settings.worker_model, settings.vision_model}

VRAM_COST_MB = {settings.worker_model: 2200, settings.vision_model: 1800}

def is_cpu(model: str) -> bool:
    return model in CPU_MODELS

```

Step 3 — llm/ollama_manager.py

The full manager. Two rules it enforces: CPU models load once and never touch the GPU, and the GPU has at most one tenant, held under a lock so two graph nodes cannot race into it.

```

# backend-ai/llm/ollama_manager.py
import asyncio, logging
import httpx
from contextlib import asynccontextmanager

from config import settings, CPU_MODELS, GPU_MODELS, VRAM_COST_MB, is_cpu

log = logging.getLogger("ultron.ollama")

try:
    import pynvml
    pynvml.nvmlInit()
    _HANDLE = pynvml.nvmlDeviceGetHandleByIndex(0)
except Exception as exc:
    # no NVML: degrade, do not crash
    log.warning("NVML unavailable (%s) – VRAM telemetry disabled", exc)
    _HANDLE = None

class OllamaManager:
    def __init__(self, bus):
        self.bus = bus
        self._gpu_tenant: str | None = None
        self._gpu_lock = asyncio.Lock()
        self._client = httpx.AsyncClient(base_url=settings.ollama_host,

```

```

                                timeout=httpx.Timeout(600.0,
connect=5.0))

# — properties —————
@property
def active(self) -> str | None:
    return self._gpu_tenant

# — primitives —————
async def _load(self, model: str, keep_alive) -> None:
    """Empty prompt = load only. num_gpu=0 pins a model to CPU."""
    opts = {"num_ctx": settings.ctx_router if is_cpu(model)
            else settings.ctx_worker}
    if is_cpu(model):
        opts["num_gpu"] = 0
    await self._client.post("/api/generate", json={
        "model": model, "prompt": "", "keep_alive": keep_alive,
"options": opts})

    async def _unload(self, model: str) -> None:
        await self._client.post("/api/generate", json={
            "model": model, "prompt": "", "keep_alive": 0})

    def _free_mb(self) -> int | None:
        if _HANDLE is None:
            return None
        m = pynvml.nvmlDeviceGetMemoryInfo(_HANDLE)
        return (m.total - m.used) // 1_048_576

# — lifecycle —————
async def warm_resident(self) -> None:
    """Router and embedder onto CPU, forever. Called from lifespan
startup."""
    for model in CPU_MODELS:
        await self._load(model, keep_alive=-1)
        log.info("resident on CPU: %s", model)
        await self.bus.publish({"type": "model",
                                "payload": {"phase": "ready", "model":
"resident"}})

    @asynccontextmanager
    async def worker(self, model: str):
        """Exclusive GPU tenancy for the duration of one agent node."""

```

```

    if model not in GPU_MODELS:
        raise ValueError(f"{model} is not a GPU model – check config")

    async with self._gpu_lock:
        if self._gpu_tenant and self._gpu_tenant != model:
            await self.bus.publish({"type": "model",
                                   "payload": {"phase": "unloading", "model":
self._gpu_tenant}})
            await self._unload(self._gpu_tenant)
            self._gpu_tenant = None
            await asyncio.sleep(0.3)          # let the driver actually
release

        if self._gpu_tenant != model:
            free = self._free_mb()
            need = VRAM_COST_MB.get(model, 2200)
            if free is not None and free < need:
                await self.bus.publish({"type": "error", "payload": {
                    "severity": "warn", "source": "vram",
                    "message": f"{free} MB free, {model} wants {need} MB
_ "
                                "Ollama will offload layers to CPU and run
slowly"}}})

            await self.bus.publish({"type": "model",
                                   "payload": {"phase": "loading", "model": model}})
            await self._load(model, keep_alive="5m")
            self._gpu_tenant = model

        await self.bus.publish({"type": "model",
                                "payload": {"phase": "ready", "model": model}})
        try:
            yield model
        finally:
            await self._unload(model)
            self._gpu_tenant = None
            await self.bus.publish({"type": "model",
                                    "payload": {"phase": "evicted", "model": model}})

# — telemetry —————
async def snapshot(self) -> dict:
    if _HANDLE is None:
        return {"used_mb": 0, "total_mb": 0, "active_model":
self._gpu_tenant}

```

```

        m = pynvml.nvmlDeviceGetMemoryInfo(_HANDLE)
        return {"used_mb": m.used // 1_048_576,
                "total_mb": m.total // 1_048_576,
                "active_model": self._gpu_tenant}

    async def telemetry(self, hz: float = 1.0) -> None:
        if _HANDLE is None:
            return
        while True:
            await self.bus.publish({"type": "vram", "payload": await
self.snapshot()})
            await asyncio.sleep(1 / hz)

    async def shutdown(self) -> None:
        for model in (*GPU_MODELS, *CPU_MODELS):
            try:
                await self._unload(model)
            except Exception:
                pass
        await self._client.aclose()

```

The 300 ms sleep after an unload is not superstition. On Windows the driver releases memory asynchronously, and loading the next model immediately sometimes sees the old allocation still held — which is how you get a spurious layer offload on a machine that had room.

Step 4 — Router on CPU

```

# backend-ai/graph/supervisor.py
from langchain_ollama import ChatOllama
from config import settings

def _router_llm():
    return ChatOllama(
        model=settings.router_model,
        base_url=settings.ollama_host,
        temperature=0,
        num_gpu=0,                                # ← the whole point: stays off the
GPU
        num_ctx=settings.ctx_router,
        keep_alive=-1,
    ).with_structured_output(Route)

```

Build it once at module import, not per call. Re-instantiating `ChatOllama` is cheap, but a fresh request without `keep_alive=-1` somewhere in the chain will let the router fall out of memory and cost you a reload on the next utterance.

Step 5 — One worker, four roles

With a single worker model there is no reason for four near-identical agent factories. One factory, varying prompt and tools:

```
# backend-ai/graph/agents/worker.py
from langgraph.prebuilt import create_react_agent
from langchain_ollama import ChatOllama
from pathlib import Path

from config import settings
from tools.registry import tools_for

def make_worker(name: str, bus, ollama):
    prompt = Path(f"llm/prompts/{name}.md").read_text(encoding="utf-8")

    async def node(state):
        async with ollama.worker(settings.worker_model) as model:
            llm = ChatOllama(model=model, base_url=settings.ollama_host,
                             temperature=0.1, num_ctx=settings.ctx_worker,
                             keep_alive="5m")
            agent = create_react_agent(llm, tools=tools_for(name),
prompt=prompt)
            out = await agent.ainvoke({"messages": state["messages"]},
                                     {"recursion_limit": 12})
            return {"messages": out["messages"][-1:]}

    node.__name__ = name                    # LangGraph uses this for
    astream_events
    return node
```

Then in `builder.py`:

```
for name in ("dev_agent", "web_agent", "os_agent", "rag_agent"):
    g.add_node(name, make_worker(name, bus, ollama))
```

Because all four share one model, consecutive hops to different agents cost **zero** swap time — the `worker()` context sees the same model already resident and skips straight to ready. That is the quiet payoff of collapsing the tiers, and it matters more than the specialisation you gave up.

`recursion_limit=12` caps tool calls within a single agent, separately from `MAX_HOPS` across the graph. A 3B model that cannot find a selector will otherwise retry until the run times out.

Step 6 — The VRAM gauge

```
// src/hud/VramGauge.jsx
import { useUltron } from "../store/ultronStore";

export default function VramGauge() {
  const { used_mb, total_mb, active_model } = useUltron((s) => s.vram);
  const pct = total_mb ? (used_mb / total_mb) * 100 : 0;
  const tone = pct > 88 ? "bg-red-400" : pct > 70 ? "bg-amber-300" : "bg-cyan-300";

  return (
    <div className="glass pointer-events-auto w-56 p-3">
      <div className="flex items-baseline justify-between">
        <span className="glass-label">vram</span>
        <span className="font-mono text-xs text-cyan-100/80">
          {used_mb}<span className="text-cyan-100/40">/<total_mb> MB</span>
        </span>
      </div>

      <div className="mt-2 h-1.5 w-full overflow-hidden rounded-full bg-white/10">
        <div className={`h-full ${tone} transition-[width] duration-500`}
          style={{ width: `${Math.min(100, pct)}%` }} />
      </div>

      <div className="mt-2 truncate font-mono text-[10px] text-cyan-200/50">
        {active_model ?? "gpu idle"}
      </div>
    </div>
  );
}
```

Watching this gauge during the first end-to-end run tells you more than any log. You should see a flat baseline, one sharp rise when the worker loads, a plateau, and a drop back to baseline — four distinct shapes. Anything else means the manager is not doing its job.

Step 7 — Verify

```
# scripts/check_vram.py – run three tasks, assert the GPU returns to idle
import asyncio, subprocess, httpx

TASKS = ["what is in my notes about the blueprint",
         "open my editor",
         "what does Gita 2.47 say"]

def ollama_ps() -> str:
    return subprocess.run(["ollama", "ps"], capture_output=True,
                          text=True).stdout

async def main():
    for t in TASKS:
        async with httpx.AsyncClient() as c:
            await c.post("http://127.0.0.1:8765/debug/run", json={"text": t})
            await asyncio.sleep(3)
            ps = ollama_ps()
            assert "qwen2.5-coder" not in ps, f"worker still resident after:
{t}\n{ps}"
            assert "qwen2.5:1.5b" in ps and "100% CPU" in ps, f"router
misplaced\n{ps}"
            print("VRAM discipline OK")

asyncio.run(main())
```

Add the `/debug/run` endpoint behind a flag — it is the same call path as a `user_utterance` frame and saves you clicking through the UI on every iteration.

Step 8 — Troubleshooting

Symptom	Cause	Fix
Worker shows 100% CPU in <code>ollama ps</code>	Free VRAM below the model size at load time	Close Chromium, raise <code>OLLAMA_GPU_OVERHEAD</code> , confirm nothing else holds VRAM
~4 tok/s generation	Partial layer offload	Same as above; check <code>nvidia-smi</code> during generation, not after
Router also on GPU	<code>num_gpu=0</code> missing on one call path	Grep for <code>ChatOllama()</code> — every router and embedder call needs it
VRAM never returns to baseline	An exception skipped the <code>finally</code>	The <code>async with</code> handles this; check no agent calls Ollama outside <code>worker()</code>
First utterance after boot takes 8 s	<code>warm_resident()</code> not awaited in lifespan	Await it before yielding, not as a background task
Gauge frozen at 0	<code>pynvml</code> import failed silently	Check the startup warning; <code>pip install nvidia-ml-py</code> , not <code>pynvml</code>
Two workers loaded at once	A node called Ollama directly	All model access goes through <code>ollama.worker()</code> — no exceptions

Exit criteria

- `ollama ps` shows exactly the CPU router between tasks.
- Peak dedicated VRAM stays under 2,900 MB with Chromium open.
- The worker sustains 30+ tok/s during generation (check with `--verbose` on a direct `ollama run`).
- Three consecutive tasks leave no resident GPU model.
- `check_vram.py` passes clean.

Hit all five and the hardware question is settled; P5 onward is ordinary application work.